MotionLab: interação gestual sem contato com ESP32, sensores de baixo custo e comunicação MQTT segura

João Henrique R. V. de Oliveira, Antônio Jose Paes de Andrade Junior, Marco Antônio Maciel, Jorge Dias, João Pedro

CESAR School — Recife, PE — Brasil
Sistemas Embarcados — Grupo 5

Repositório: github.com/jhrvo0/ChickenLittle

Abstract. *MotionLab is a low-cost embedded system for touchless gesture interaction. An ESP32 running FreeRTOS reads an HC-SR04 ultrasonic sensor and a TCRT5000 infrared sensor every 70 ms and delivers samples over two paths: USB serial with a strict JSON Lines protocol, which feeds a local web dashboard with five interactive modules, and Wi-Fi with MQTT over TLS, which feeds a broker and an isolated database. On the bench, the board delivered 432 samples in 30 s at the nominal period with no new queue drops in that window and reconnected after a broker restart. A reproducible machine learning pipeline compares a learned baseline, a rule-based reference and three classifiers; with synthetic data the compact MLP reached macro-F1 0.97 on a held-out participant. These figures validate the pipeline only; physical gesture accuracy remains to be measured.*

Resumo. *O MotionLab é um sistema embarcado de baixo custo para interação gestual sem contato. Uma ESP32 com FreeRTOS lê um sensor ultrassônico HC-SR04 e um sensor infravermelho TCRT5000 a cada 70 ms e entrega as amostras por dois caminhos: USB serial com protocolo JSON Lines estrito, que alimenta um dashboard web local com cinco módulos interativos, e Wi-Fi com MQTT sobre TLS, que alimenta um broker e um banco isolado. Em bancada, a placa entregou 432 amostras em 30 s no período nominal, sem novos descartes na fila nessa janela, e reconectou após o reinício do broker. Um pipeline reprodutível de aprendizado de máquina compara um baseline aprendido, uma referência por regras e três classificadores; com dados sintéticos, o MLP compacto obteve F1-macro de 0,97 em um participante reservado. Esses números validam apenas o pipeline; a precisão física dos gestos ainda precisa ser medida.*

1. Introdução

Interfaces sem contato são úteis quando tocar no equipamento é inconveniente: mãos ocupadas ou sujas, ambientes que exigem higiene, apresentações em que o orador está longe do teclado e atividades lúdicas ou de acessibilidade. Soluções comerciais costumam depender de câmeras ou de sensores de profundidade, que custam mais e levantam questões de privacidade. Este trabalho investiga até onde é possível chegar com dois sensores baratos e amplamente disponíveis em laboratórios de ensino: um sensor ultrassônico de distância e um sensor infravermelho de reflexão.

O resultado é o **MotionLab**, um laboratório de interação gestual composto por firmware embarcado, comunicação por USB e por MQTT, um dashboard web e um

pipeline de aprendizado de máquina. O projeto nasceu como um bichinho virtual controlado por gestos (MotionPet) e evoluiu para uma plataforma mais geral; a experiência original continua disponível como módulo legado.

Os objetivos foram: (i) adquirir distância e presença com temporização determinística em uma ESP32; (ii) transmitir os dados ao notebook por USB, para a interação ao vivo, e por Wi-Fi/MQTT com autenticação e criptografia, para coleta e processamento; (iii) exibir os dados em tempo real em uma interface visual; (iv) reconhecer gestos por regras e por modelos treinados, comparando-os com metodologia que evite vazamento de dados; e (v) registrar com honestidade o que foi validado fisicamente e o que foi validado apenas em software.

As seções seguintes apresentam a fundamentação (Seção 2), a metodologia e a arquitetura (Seção 3), os resultados (Seção 4), a discussão das limitações (Seção 5), a conclusão (Seção 6) e a divisão de tarefas (Seção 7). Trechos de código estão nos apêndices.

2. Fundamentação

O HC-SR04 mede distância pelo tempo de voo de um pulso ultrassônico de 40 kHz; fornece um único valor escalar por leitura, com cone de abertura estreito, e por isso não distingue posição lateral nem pose da mão. O TCRT5000 combina um LED e um fototransistor infravermelhos e, no módulo usado, entrega uma saída digital de presença próxima. A combinação dos dois permite descrever gestos temporais de aproximação, afastamento e permanência, além de toques rápidos de presença.

A ESP32 é um microcontrolador de dois núcleos com Wi-Fi e Bluetooth integrados [Espressif Systems 2023]. O firmware usa FreeRTOS para separar aquisição, inferência e comunicação em tarefas com prioridades distintas [Barry 2016]. Para a comunicação em rede foi adotado o MQTT 3.1.1, protocolo publicação/assinatura leve e padronizado pela OASIS [OASIS 2014], com o broker Mosquitto [Light 2017]. O nível de serviço QoS 1 garante entrega pelo menos uma vez por meio de confirmações (PUBACK); a confidencialidade e a autenticação do servidor são obtidas com TLS [Rescorla 2018].

No reconhecimento de gestos, modelos compactos como árvores de decisão, florestas aleatórias [Breiman 2001] e redes neurais pequenas podem ser executados no host e, depois de convertidos, no próprio microcontrolador, abordagem conhecida como TinyML [Warden e Situnayake 2019]. O treinamento usa scikit-learn [Pedregosa et al. 2011] e o rastreamento de experimentos usa MLflow [Zaharia et al. 2018], orquestrado por um DAG do Apache Airflow [Apache Software Foundation 2024]. Como janelas temporais sobrepostas e amostras do mesmo participante são correlacionadas, a validação precisa agrupar os dados por participante e reservar o teste antes da seleção de modelos, para não superestimar o desempenho [Varma e Simon 2006].

3. Metodologia e arquitetura

3.1. Hardware

A montagem usa uma ESP32 DevKit de 38 pinos, um HC-SR04, um TCRT5000, três resistores e duas meias-protoboards, alimentados pela USB do notebook (Figuras 1 e 2). O HC-SR04 opera em 5 V; como os GPIOs da ESP32 suportam apenas 3,3 V, o sinal

ECHO passa por um divisor resistivo (1 kΩ em série; 1 kΩ + 300 Ω para o terra), cuja saída nominal calculada é de 2,83 V, antes de chegar ao GPIO34. O TRIG é acionado pelo GPIO5. O TCRT5000 é alimentado em 3,3 V e sua saída digital chega ao GPIO35. Os trilhos de 5 V e 3,3 V são separados e os terras são comuns.

Um display LCD 2004 I2C foi testado em versões anteriores e retirado: em 3,3 V o texto ficava ilegível e, em 5 V, o barramento I2C exigiria um conversor de nível lógico. A interface web no notebook assumiu a função de exibição. A montagem vigente está registrada em uma netlist, em um diagrama Wokwi e em um guia de ligações versionados no repositório.

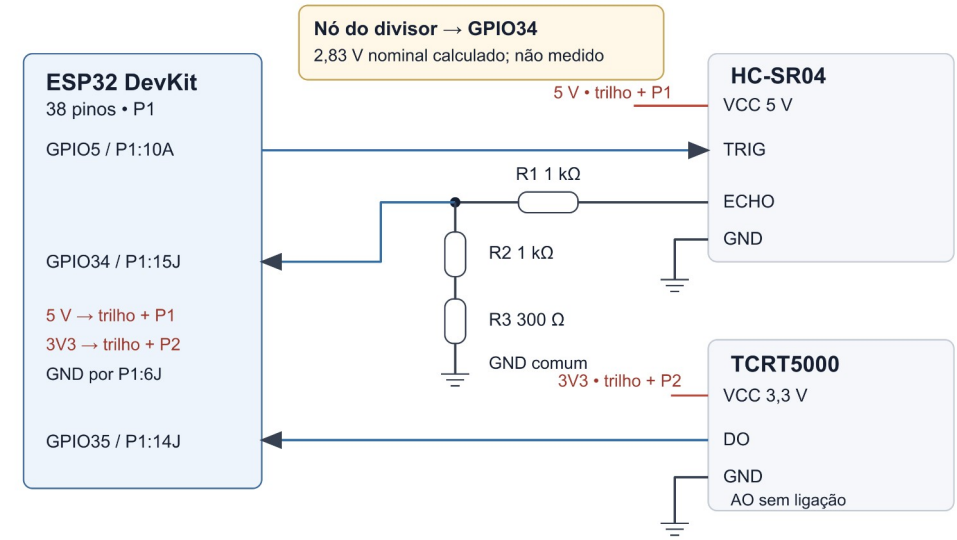
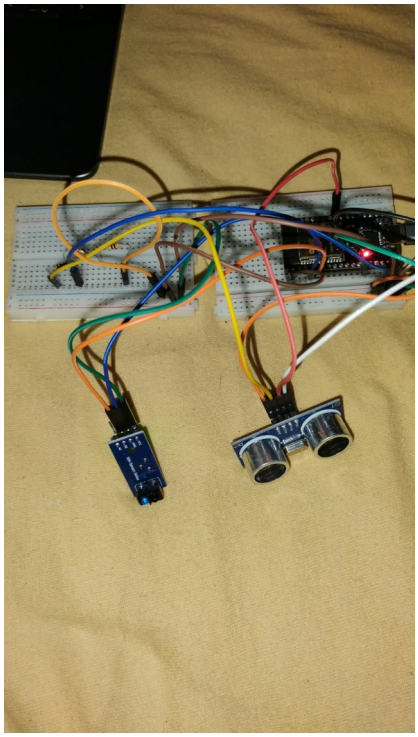
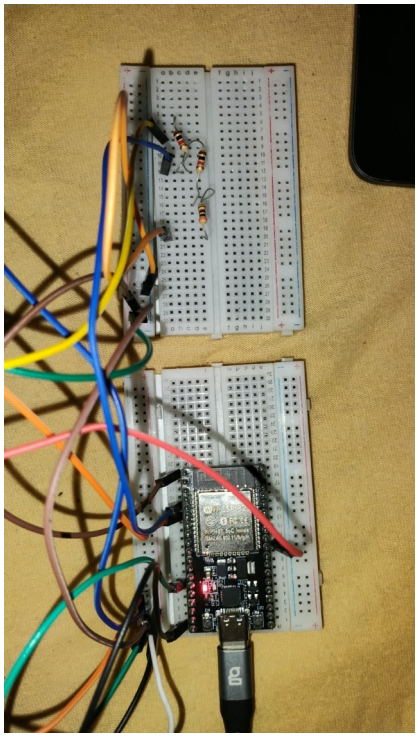


Figura 1. Circuito vigente, derivado da netlist da montagem (sem LCD)



(a) conjunto com os sensores



(b) ESP32 e divisor do ECHO

Figura 2. Fotografias da montagem na bancada

3.2. Firmware

O firmware foi escrito em C++ com o framework Arduino sobre FreeRTOS e compilado com PlatformIO (ambiente `esp32dev`). As tarefas são fixadas no núcleo de aplicação: a tarefa de aquisição (prioridade mais alta) dispara o ultrassom e lê o infravermelho a cada 70 ms; a de inferência aplica um filtro causal, mantém um buffer circular, extrai 11 características e executa o modelo TinyML com um estabilizador de eventos (confiança mínima de 0,60, duas janelas consecutivas e 700 ms de bloqueio); a de comunicação é a única dona da UART e implementa o protocolo serial; uma tarefa de domínio mantém o estado do modo legado. As mensagens internas usam filas estáticas, sem alocação dinâmica no caminho crítico.

O transporte IoT é opcional e configurado em tempo de compilação por um cabeçalho privado, ignorado pelo Git, com SSID, credenciais e certificado da autoridade certificadora. Sem esse arquivo, o firmware funciona apenas por USB e o IoT fica desativado. Quando habilitado, a tarefa de aquisição enfileira cada amostra em uma fila estática de 32 posições com política *drop-new* (descarta a amostra nova quando cheia e conta o descarte); uma tarefa dedicada mantém a máquina de estados de conexão Wi-Fi e MQTT e publica uma amostra por vez, só retirando-a da fila após o PUBACK. Assim, uma falha nunca bloqueia a aquisição nem o USB.

3.3. Comunicação

USB serial. O protocolo JSON Lines v1 envia um objeto JSON por linha, a 115200 baud, com envelope versionado (`v`, `seq`, `type`, `ts_ms`, `data`). Os tipos principais são `sensor_frame` (distância, validade e IR), `gesture_event` (classe, confiança e

origem) e comandos do host com confirmação (ACK/ERROR) pelo número de sequência. O parser do firmware é estrito, opera em buffer fixo e rejeita mensagens malformadas.

Wi-Fi/MQTT. Cada amostra é publicada com QoS 1 no tópico `motionlab/v1/devices/<device_id>/samples`. O payload (Apêndice A) traz identificador do dispositivo derivado do MAC, identificador de sessão por inicialização, índice da amostra, carimbo de tempo monotônico, leituras com validade explícita, origem declarada e contador acumulado de descartes. O broker Mosquitto roda no WSL do notebook com TLS, autenticação por usuário e senha e ACL que restringe cada dispositivo ao próprio tópico; um encaminhador expõe a porta TLS apenas para a rede local privada. Um consumidor Python valida o esquema, remove duplicatas, rejeita origens não autorizadas e persiste as amostras em um banco SQLite isolado. O campo `physical_validation` é sempre `pending`: o dispositivo não pode atestar a própria validação.

3.4. Aplicação web e reconhecimento

O backend Python (porta 8767) lê a serial, mantém a sessão e executa o *MotionEngine*, que roda em paralelo três reconhecedores: regras determinísticas (padrão), o modelo de ML no host e os eventos TinyML vindos da placa. Um portão de segurança decide quais eventos acionam os módulos; ações externas (teclas) usam um catálogo fechado, ficam desligadas a cada início e têm limite de 8 teclas a cada 10 s. O frontend, em HTML, CSS e JavaScript sem frameworks, consulta `/api/live` e oferece cinco módulos: *Motion Control* (ajustes e perfis), *Motion Canvas* (desenho com a distância), *Air Music* (notas e timbres), *Air Slides* (controle de apresentações) e *Motion Intelligence* (sinais ao vivo, modelos e inferências). Um modo DEMO usa um simulador determinístico e roda sem a placa; toda tela marca a proveniência dos dados (REAL, SINTÉTICO, DEMO ou PENDENTE). A Figura 3 resume a arquitetura.

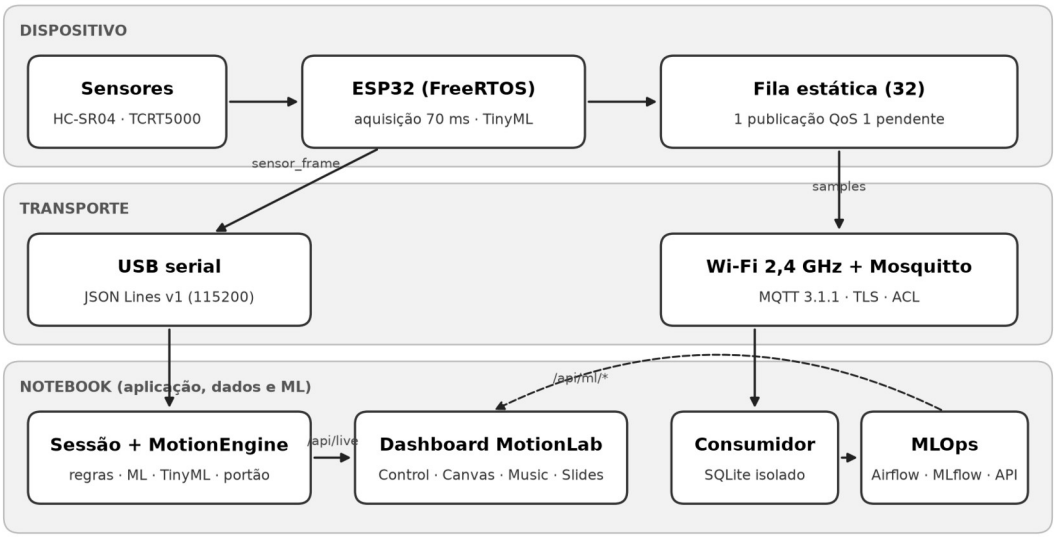


Figura 3. Arquitetura do MotionLab: caminhos USB (interação ao vivo) e Wi-Fi/MQTT (coleta)

3.5. Aprendizado de máquina e MLOps

As classes do contrato são NEUTRAL, APPROACH, RETREAT e HOLD. As amostras passam por filtro causal com estado isolado por sessão e são agrupadas em janelas, das quais se extraem 11 características (estatísticas de distância, velocidade, tendência e IR). O experimento reserva o participante de teste antes de qualquer análise exploratória, aplica GroupKFold por participante no desenvolvimento, ajusta o *StandardScaler* somente dentro de cada fold e compara: uma árvore de profundidade 1 como baseline aprendido; as regras como referência de domínio; uma árvore de decisão; uma floresta aleatória; e um MLP compacto. A seleção usa apenas a validação, combinando F1-macro e custo estimado de memória; o teste reservado é avaliado uma única vez depois da escolha.

Cada execução registra hash do corpus, instantâneo do código, configuração, sementes e métricas no MLflow. Um DAG do Airflow com quatro tarefas orquestra a preparação, o treino, a avaliação e o registro em lote; a publicação de um modelo para a API de inferência (porta 8768) é sempre explícita, acompanhada dos hashes. O modelo escolhido é exportado como cabeçalho C++ e verificado por um teste de paridade numérica entre Python e C++.

3.6. Procedimentos de ensaio

O ensaio de comunicação usou a placa real em uma rede Wi-Fi residencial de 2,4 GHz, com o broker no notebook. Antes de gravar o firmware IoT, fez-se um backup integral da memória flash (4 MB), conferido pelo esptool. Foram medidas, em janelas de 20 a 30 s, as amostras persistidas no banco e a variação do contador de descartes informado pela placa, comparando versões do firmware e períodos de aquisição; por fim, o broker foi reiniciado brevemente para observar a reconexão. A avaliação de ML foi feita com um corpus sintético de cinco participantes simulados, gerado pelo simulador do projeto. O software é verificado por 658 testes automatizados em Python e 20 testes de interface em Node.js.

4. Resultados

4.1. Montagem e leitura dos sensores

A montagem da Figura 2 opera com os dois sensores lendo simultaneamente e sem erros de leitura reportados, tanto pelo USB quanto pelo MQTT; em uma das amostras persistidas, o ultrassom e o IR estavam ambos válidos. Não foram feitas medições de tensão nem de precisão da distância, porque a equipe não dispunha de multímetro nem de gabarito de referência; o valor de 2,83 V no nó do divisor é calculado.

4.2. Comunicação Wi-Fi/MQTT

Na primeira tentativa, a biblioteca TLS da ESP32 recusou o certificado do servidor, que identificava o broker apenas pelo nome. O provisionador passou a gerar certificados com o endereço tanto em SAN de IP quanto em SAN de nome, mantendo a mesma CA e a verificação do servidor ativa, e a placa conectou. A Tabela 1 mostra a evolução da vazão.

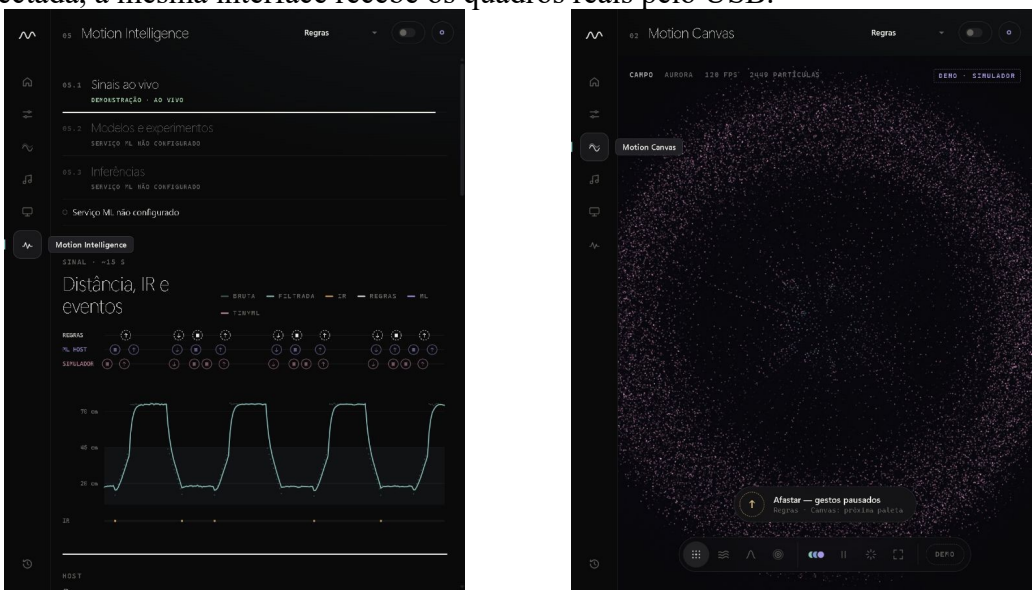
Tabela 1. Amostras persistidas no banco e descartes na fila da ESP32

Firmware e período	Janela	Amostras	Novos descartes
Versão inicial, 70 ms	20 s	183	80
Versão inicial, 150 ms (temporário)	25 s	183	0
Polling ajustado, 70 ms	30 s	394	40
Polling ajustado, 90 ms (temporário)	30 s	334	0
Polling ajustado + modem-sleep desligado, 70 ms	30 s	432	0
Mesma versão, após reinício do broker	20 s	291	0

Dois ajustes resolveram o gargalo: reduzir a espera da tarefa de rede entre a confirmação e a próxima publicação (o polling passou de 20 ms para 1 ms por ação quando conectada) e desativar a economia de energia do rádio Wi-Fi (modem-sleep), que atrasava as confirmações. Com ambos, a placa sustentou o período nominal de 70 ms (cerca de 14 amostras por segundo; 432 em 30 s) sem novos descartes na janela, com fila em 1/32 e nenhum erro de publicação. Depois do reinício do broker, a placa voltou ao estado *online* sozinha e o consumidor continuou recebendo. O contador acumulado da sessão registrou 89 descartes, que incluem as tentativas iniciais e o intervalo do reinício.

4.3. Interface em tempo real

O dashboard exibe a distância bruta e filtrada, o estado do IR e os eventos de cada reconhecedor em linhas separadas, atualizados continuamente (Figura 4). As capturas foram feitas no modo DEMO, com o simulador, e estão marcadas como tal; com a placa conectada, a mesma interface recebe os quadros reais pelo USB.



(a) Motion Intelligence: sinais ao vivo (b) Motion Canvas

Figura 4. Capturas da interface MotionLab em modo DEMO

4.4. Avaliação dos modelos (dados sintéticos)

O experimento de referência usou 896 amostras, 395 janelas e cinco participantes simulados: três para treino, um para validação e um reservado para teste. O DAG executou as quatro tarefas com sucesso em 50 s, e o MLflow registrou oito execuções filhas e cinco modelos. A Tabela 2 apresenta o F1-macro sobre as quatro classes.

Tabela 2. F1-macro por modelo no corpus sintético (CV com quatro folds)

Modelo	Treino	Validação	Teste	CV (média ± desvio)
Árvore prof. 1 (baseline)	0,339	0,328	0,355	0,338 ± 0,008
Regras (referência)	0,506	0,525	0,572	0,509 ± 0,013
Árvore de decisão	0,931	0,738	0,596	0,715 ± 0,036
Floresta aleatória	0,972	0,873	0,847	0,806 ± 0,102
MLP compacto	0,993	0,896	0,971	0,856 ± 0,044

O baseline de profundidade 1, com apenas duas folhas para quatro classes, subajusta como esperado e serve de piso. A árvore de decisão mostra sinal de sobreajuste: queda de cerca de 0,19 do treino para a validação e nova queda no teste. A floresta reduz essa queda, mas varia mais entre os folds. O MLP foi escolhido pela validação e pelo custo; o fato de ir melhor no teste do que na validação indica apenas que o participante reservado foi mais favorável nessa partição, e não autoriza uma nova escolha com base no teste.

4.5. Verificação de software

A suíte Python termina com 655 testes aprovados e 3 pulados (dependem de peças do Fritzing não instaladas), cobrindo protocolo, reconhecedores, portão de ações, ingestão MQTT, provisionamento do broker, pipeline de ML e API. Os 20 testes de interface em Node.js passam, e o firmware com IoT habilitado compila usando 17,0% da RAM (55,6 kB) e 71,6% da flash de aplicação (938 kB), a maior parte ocupada pelas pilhas Wi-Fi e TLS.

5. Discussão e limitações

- **Dados sintéticos.** As métricas da Tabela 2 vêm de cinco participantes simulados por um gerador próximo das próprias regras; elas comprovam que o pipeline funciona, não que os modelos generalizam para pessoas reais. Por isso, a fonte padrão de gestos da interface continua sendo as regras.
- **Janelas curtas.** Os ensaios sem novos descartes cobrem 20 a 30 s. A fila é volátil, então quedas longas de rede perdem amostras; faltam ensaios longos, quedas prolongadas e reinícios da placa.
- **Consumo.** Desligar o modem-sleep aumenta o consumo do rádio, o que não importa com alimentação USB, mas importaria com bateria.
- **Rede.** O certificado inclui o IP do notebook no momento do provisionamento; se o IP mudar, é preciso reemitir o certificado, sem desativar a verificação TLS. O projeto usa IPv4; IPv6 não foi implementado.
- **Sensor.** O ultrassom fornece um valor escalar, de modo que gestos laterais ou de pose não podem ser distinguidos sem sensores adicionais.

6. Conclusão e trabalhos futuros

O MotionLab mostra que dois sensores de baixo custo e uma ESP32 são suficientes para uma plataforma completa de interação gestual: aquisição temporizada com FreeRTOS, dois caminhos de comunicação (USB para a interação ao vivo e MQTT com TLS para a coleta), dashboard em tempo real com cinco módulos e um pipeline de

ML reprodutível e auditável. O ensaio de bancada confirmou o envio por Wi-Fi/MQTT no período nominal e a reconexão após falha do broker.

Como próximos passos, a equipe pretende: coletar gestos reais de participantes com protocolo e consentimento, para avaliar regras e modelos fora do cenário sintético; executar ensaios longos de rede com quedas prolongadas; levar os sinais recebidos por MQTT ao painel ao vivo; e explorar funcionalidades extras como alertas por mensagem e comunicação em IPv6.

7. Divisão de tarefas

Tabela 3. Responsabilidades dos integrantes do Grupo 5

Integrante	Responsabilidade
João Henrique R. V. de Oliveira	Representação do grupo e coordenação
Antônio Jose Paes de Andrade Junior	Hardware, sensores e montagem
Marco Antônio Maciel	Aprendizado de máquina e avaliação dos modelos
Jorge Dias	Interface e experiências interativas
João Pedro	Comunicação e integração Wi-Fi/MQTT

Uso de inteligência artificial. Conforme autorizado pelo professor, a equipe usou assistentes de IA no desenvolvimento do código, na documentação e na redação deste relatório. Os resultados relatados vêm de execuções registradas no repositório e dos ensaios de bancada descritos.

Referências

Apache Software Foundation (2024). *Apache Airflow Documentation*. Disponível em: https://airflow.apache.org/docs/.

Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Espressif Systems (2023). *ESP32 Series Datasheet*. Disponível em: https://www.espressif.com/en/support/documents/technical-documents.

Light, R. A. (2017). Mosquitto: server and client implementation of the MQTT protocol. *Journal of Open Source Software*, 2(13):265.

OASIS (2014). *MQTT Version 3.1.1*. OASIS Standard. Editado por A. Banks e R. Gupta.

Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF.

Varma, S. e Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91.

Warden, P. e Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

Zaharia, M. et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.

Apêndice A. Trechos de código

A.1. Quadro serial (JSON Lines v1) enviado pela ESP32.

```
{"v": 1, "seq": 104, "type": "sensor_frame", "ts_ms": 15420,
 "data": {"distance_cm": 18.4, "ir_detected": false,
          "valid_ultrasonic": true}}
```

A.2. Amostra publicada por MQTT (tópico `motionlab/v1/devices/<device_id>/samples`).

```
{"schema_version":1,"kind":"sample","device_id":"esp32_112233aabbcc",
 "session_id":"boot_001122","sample_idx":0,"ts_ms":1000,
 "data_origin":"real_hardware","distance_cm":40.00,
 "valid_ultrasonic":true,"ir_detected":false,"valid_ir":true,
 "captured_at_iso":null,"clock_source":"monotonic_boot",
 "physical_validation":"pending","dropped_total":0}
```

A.3. Publicação QoS 1 na tarefa de rede (`firmware/communication/iot_transport.cpp`).

```
case IOT_ACK_SAMPLE:
    portENTER_CRITICAL(&g_lock);
    iot_queue_ack(&g_queue);
    portEXIT_CRITICAL(&g_lock);
    break;
case IOT_PUBLISH_SAMPLE: {
    // Uma publicação QoS 1 pendente; a amostra sai da fila só após o PUBACK.
    const bool serialized = iot_serialize_sample(&sample, g_device, g_session,
                                                 g_payload, sizeof(g_payload));
    const int mid = serialized
        ? esp_mqtt_client_publish(g_client, g_topic, g_payload, 0, 1, 0) : -1;
    iot_connection_published(&g_connection, now_ms, mid);
    if (mid < 0) {
        portENTER_CRITICAL(&g_lock); g_broker_failed = true; ++g_publish_errors;
        portEXIT_CRITICAL(&g_lock);
    }
    break;
}
```

A.4. Reconhecedor no host (`terminal/motionlab/recognizers.py`).

```
def push(self, frame, ts_ms):
    if self.classifier is None:
        return None
    before = self.classifier.get_diagnostics()
    event = self.classifier.push_frame(frame, ts_ms=ts_ms)
    diag = self.classifier.get_diagnostics()
    window = self.bundle.preprocessing_config.window_size
    if diag.samples_used >= window:
        self.stats.windows += 1
        self.stats.latencies_ms.append(diag.last_latency_ms)
        reason = diag.last_rejection_reason
        if diag.rejected_events > before.rejected_events and reason:
            self.stats.rejected[reason] = self.stats.rejected.get(reason, 0) + 1
    if event is not None:
        self.stats.events += 1
    return event
```

Apêndice B. Organização do repositório

Tabela 4. Principais diretórios do repositório

Diretório	Conteúdo
firmware/	Firmware ESP32 (FreeRTOS, PlatformIO), sensores, protocolo e IoT
terminal/	Backend Python, MotionEngine e interface web (applications/)
ml/ · simulator/	Pipeline de ML, API de inferência, ingestão MQTT e simulador
ops/	Broker Mosquitto com TLS/ACL e stack MLOps (Airflow, MLflow)
hardware/	Netlists, diagrama Wokwi e guias de montagem (schematics/)
docs/	Arquitetura, protocolos, ensaios, fotos, capturas e este relatório

Diretório	Conteúdo
tests/	658 testes Python e testes de interface em Node.js